INTERVIEW STUDY GUIDE · VOLUME 4

Interview Preparation and Question Bank

Concise explanations, difficult follow-ups, whiteboard flows, STAR stories, résumé wording, and a defensible fact sheet.

AM13E230 TINIElite Edge AI · Jainil Bavishi · 23 October 2026

Contents

1. How to prepare
2. Opening explanations
3. Whiteboard walkthroughs
4. Technical question bank
5. Hard challenges and honest answers
6. STAR stories
7. Résumé wording and ownership
8. Live demonstration plan
9. Final fact sheet

1. How to prepare

1. **Start with Volume 0:** understand Kconfig, object code, static libraries, and out-of-tree work before memorizing project terms.
2. **Master Volume 1:** explain the project without jargon.
3. **Draw the data path:** host → protocol → Zephyr → generated runtime → NPU interrupt → result.
4. **Own five deep stories:** clock mismatch, NPU IRQ, protocol recovery, watchdog, WSL migration.
5. **Know boundaries:** supplied model versus your integration work; regression versus accuracy.
6. **Practice numbers last:** understand what each metric measures before memorizing it.

> **INTERVIEW RULE**
> Never bluff about binary model internals or hardware behavior you did not measure. A precise boundary—followed by how you verified the available interface—is a stronger engineering answer than an invented detail.

2. Opening explanations

2.1 Ten-second answer

“It is a Zephyr-based edge-AI system that runs a supplied waveform-classification model on TI's AM13E230 TINIElite NPU, with custom platform drivers, a robust UART protocol, fault recovery, WSL tooling, and hardware-backed validation.”

2.2 Sixty-second answer

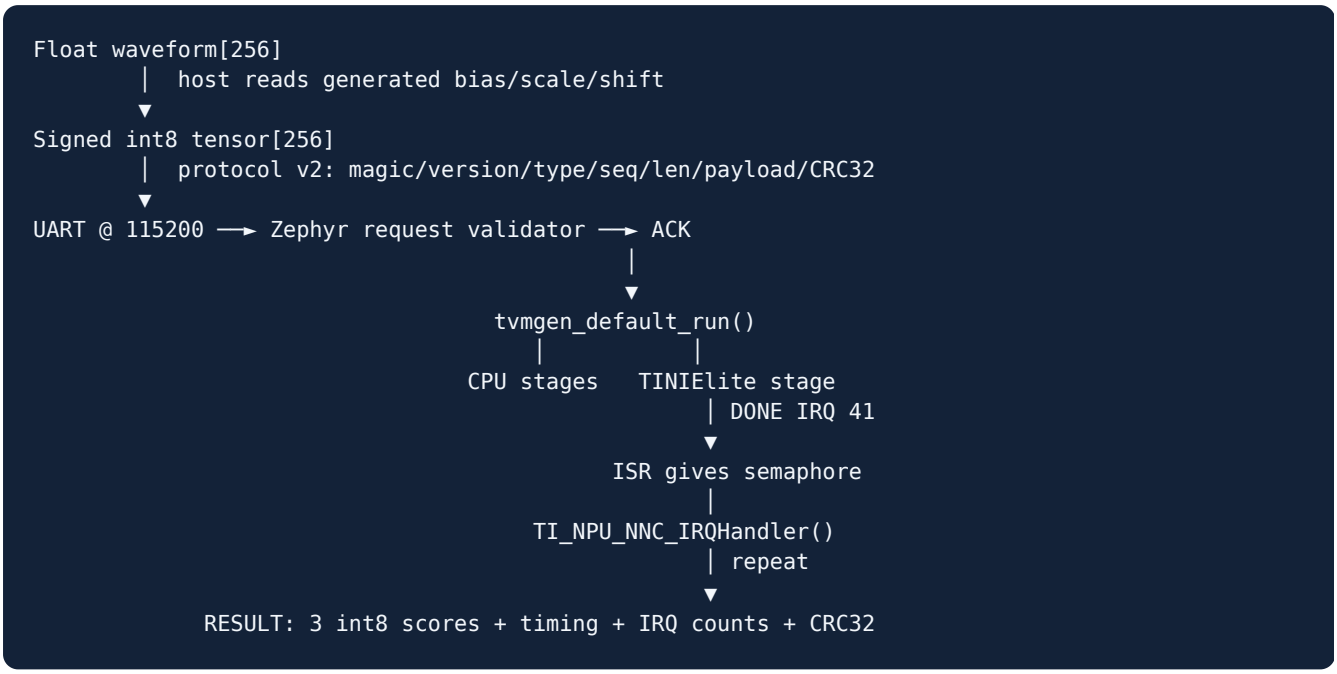
“The board was a preview TI microcontroller with a Cortex-M33 and TINIElite NPU, but the pinned Zephyr release did not support it. I built an out-of-tree board/SoC integration, configured the real 200 MHz clock, and implemented pinctrl, a polling UNICOMM UART driver, and the NPU adapter required by a supplied TVM-generated model archive. A Python client quantizes 256 waveform samples and sends them using a versioned CRC-32 UART protocol. The generated runtime executes a hybrid graph; the NPU raises IRQ 41 after hardware stages, a minimal ISR signals a semaphore, and the application continues the runtime in thread context with a timeout. I added cold-reset recovery, an independent hardware watchdog, WSL-native build/flash/test, and CI. Hardware stress testing completed 600 exact fixture inferences with 3,000 acknowledged NPU interrupts, zero spurious interrupts, and zero retries. I integrated the model; I did not train it, and the six fixtures are not a general accuracy claim.”

2.3 “What was difficult?” answer

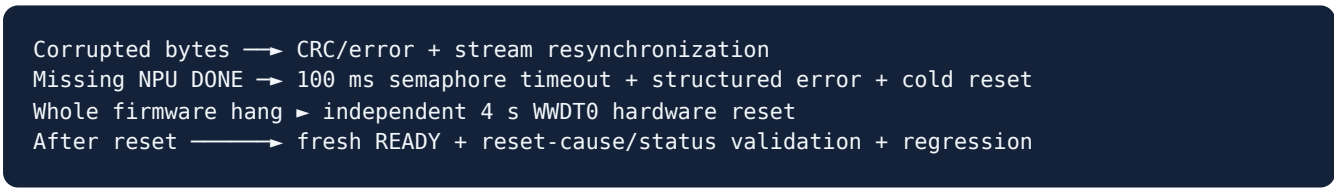
“The difficult part was establishing trustworthy behavior across several opaque boundaries: a preview MCU, sparse platform support, and a binary-only model runtime. The key was to replace assumptions with evidence—inspect archive symbols, validate actual clock registers, count physical NPU interrupts, inject failures, and preserve machine-readable hardware results.”

3. Whiteboard walkthroughs

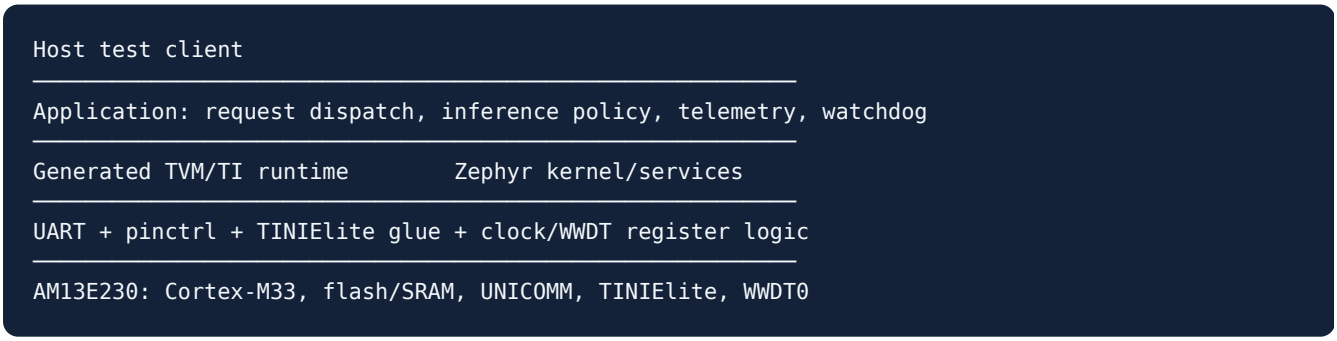
3.1 Data path



3.2 Reliability layers



3.3 Software layering



4. Technical question bank

Project and AI fundamentals

1. What makes this an edge-AI project?
Inference runs on the microcontroller/NPU near the data source. The host supplies and verifies test inputs, but no cloud service computes the model output.

2. Did you train the model?
No. TI tooling supplied a generated static archive, header, and six fixtures. My work was platform bring-up, runtime integration, communications, reliability, tooling, and validation.

3. What exactly is classified?
A vector of 256 waveform features/samples. The model returns three signed int8 scores ordered sawtooth, sine, square; the largest score is treated as the winner.

4. Are the outputs probabilities?
No. They are signed quantized scores. There is no demonstrated softmax calibration or probability interpretation.

5. What is quantization?
Mapping higher-precision values into a small integer range. Here the host applies generated bias, scale, integer shift, and clipping to produce int8 input, reducing memory and matching the accelerator's compiled model contract.

6. Why must preprocessing match exactly?
The model was compiled for a particular numeric transformation. Different scaling, rounding, clipping, ordering, or signedness changes tensor values and can change predictions even if firmware and model are otherwise correct.

7. Why is this a hybrid graph?
The generated runtime orchestrates the graph on Cortex-M33, executes some generated CPU operations, and offloads supported convolution/pooling work to TINIElite.

Zephyr and platform bring-up

8. Why use Zephyr instead of bare metal?
Zephyr provides startup, device registration, devicetree/Kconfig integration, kernel synchronization, time APIs, reboot support, native tests, and a standard build ecosystem. Low-level drivers were still required for unsupported hardware.

9. What does out-of-tree mean?
“Tree” means the directory tree of the upstream Zephyr repository. Out-of-tree means our board, SoC, devicetree, bindings, and drivers live in *our* repository instead of editing files inside `zephyr/`. `zephyr/module.yml` tells a clean pinned Zephyr checkout to discover them during the build. This makes the project reproducible: cloning Zephyr plus this repository is enough; no hidden local Zephyr edits are required. It does not mean the code is outside the final firmware—Zephyr includes it in the build.

10. What is west?

Zephyr's multi-repository workspace tool. It reads the manifest, fetches exact dependencies, locates the workspace, and drives commands such as build and Twister.

11. What is devicetree?

A build-time description of hardware resources and connections: memory ranges, register blocks, interrupts, clocks, pins, and enabled devices. Drivers obtain validated constants from it.

12. What is a devicetree binding?

A YAML schema for a compatible device. It documents and validates required properties, such as UART register regions and clock reference.

13. What is Kconfig?

Kconfig is Zephyr's compile-time feature-selection system. Kconfig files describe allowed options and dependencies; `prj.conf` and board defaults request values; Zephyr resolves them into a final configuration used to build the firmware. It decides whether code/features such as drivers, serial support, reboot, stack sizes, and test-only fault interfaces exist in the flashed image. It is not a runtime switch sent over UART. In this project, the NPU/watchdog fault-injection symbols are enabled for validation and should be disabled in a deployment image.

14. How do CMake, Kconfig, and devicetree differ?

CMake answers "which source files and libraries are compiled/linked?"—for example, app C files and `mod.a`. Kconfig answers "which software options are included?"—for example, serial support and test-only fault injection. Devicetree answers "what physical hardware exists and where?"—for example, UART addresses, NPU IRQ 41, pins, clocks, and watchdog registers. CMake picks the ingredients, Kconfig picks optional software features, and devicetree describes the board they run on.

15. What is MMIO?

Memory-mapped I/O exposes peripheral registers at fixed addresses. A 32-bit read/write becomes a hardware register operation. Zephyr's `sys_read32/sys_write32` make that intent explicit.

Clocking and timing

16. Why was clock configuration important?

CPU time conversion, UART baud, NPU timing, and flash safety all depend on the real clock. Merely declaring 200 MHz while hardware remained at 32 MHz created false performance numbers.

17. How is 200 MHz generated?

From the board's 25 MHz crystal using PLL predivide 2, multiply 32, and output divide 2: 25/2×32/2 = 200 MHz, followed by a staged clock-source transition.

18. Why are flash wait states required?

Flash cannot supply instructions at a 200 MHz core rate without extra access cycles. Three wait states are configured before raising the clock.

19. Why is flash configuration code in RAM?

It is unsafe to change flash access timing while executing the same operation from flash. A RAM-resident routine avoids that dependency.

20. How did you validate the clock?
Firmware reads raw source, PLL, divider, status, and flash registers, derives the active frequency, and returns both values and validation flags to the host. The evidence records 200 MHz and three wait states.

21. How are times measured?
Using the 32-bit hardware cycle counter around defined code/wait regions, then converting the unsigned cycle difference using Zephyr's known cycles-per-second value.

UART and protocol

22. Why does the UART have two base addresses?
UNICOMM has a wrapper for reset/power/mode and a UART register region for data/control/status. Applying offsets to the wrong region can access invalid addresses.

23. Is UART interrupt-driven?
No. Byte transfer uses Zephyr's polling API. NPU completion is interrupt-driven. An earlier UART IRQ attempt used an incorrect base and was reverted; async UART remains future work.

24. Why use a custom binary protocol?
The application needs exact int8 tensors and structured binary metrics. A framed protocol avoids text parsing overhead and adds length, sequencing, integrity, errors, and recovery rules.

25. Why CRC-32 rather than a checksum?
An additive checksum has weak error detection and previously exposed signedness differences. Standard CRC-32 detects common burst and multibit corruption much more reliably.

26. Why include sequence numbers?
A delayed ACK/result from an earlier retry must not satisfy a newer request. The host accepts only the expected sequence.

27. How does resynchronization work?
Both sides scan for two magic bytes. The host keeps a sliding buffer and, after an invalid candidate, drops one byte and searches again so it can recover even when a new frame begins inside a damaged candidate.

28. What is the purpose of READY?
It means the server is idle and communicates expected input/output sizes. It also provides a clear post-reset synchronization point.

29. What is the purpose of ACK?
It confirms a particular sequence passed framing and CRC checks and was accepted before the longer inference or fault action begins.

NPU integration and interrupts

30. What is `mod.a`?
A static archive generated by TI MCU NNC/TVM for Cortex-M33 and TINIElite. It contains graph/runtime code and generated accelerator kernels, linked into the final ELF.

30a. What is object code, and why is `mod.a` important?

Object code is processor-specific binary produced when the compiler translates a C file. A `.a` static library is an archive containing object code. `mod.a` is supplied compiled model/runtime code, not model source. At link time, the linker combines required archive members with Zephyr, drivers, and application object files into `zephyr.elf`; CI checks symbols in that final ELF. The flashable `zephyr.hex` is then derived from the ELF. This is why the project integrated a supplied model rather than training or dynamically loading one.

31. What is an ABI?

The binary contract between separately compiled components: symbol names, parameter types, calling convention, layouts, and behavior. Our NPU callbacks must exactly match what `mod.a` expects.

32. What callbacks does the archive need?

`NPU_getCtlReg`, `NPU_setInstrMem`, and `NPU_setParamMem`. They expose the control address and safely stage generated words into NPU SRAM.

33. Why not keep all weights in NPU SRAM?

Parameter SRAM is only 2 KiB. The generated runtime uses it as a stage-local scratchpad and swaps content as the graph progresses.

34. How does NPU completion work?

TINIElite raises DONE on IRQ 41. The ISR reads masked status, clears DONE, increments an atomic count, and gives a semaphore. The waiting thread wakes and calls the generated continuation function.

35. Why not call the generated handler inside the ISR?

Opaque generated code may be long or use services unsuitable for interrupt context. A minimal ISR reduces latency and defers graph continuation to normal thread context.

36. Why use an atomic counter?

The ISR writes the counter while thread context reads it. Atomic operations avoid a data race and ensure each completion is counted safely.

37. What is a spurious interrupt?

An entry into the TINIElite ISR when masked DONE is not set. It may indicate misrouting, stale state, or another unexpected source. The driver counts it and valid results require zero.

38. Why five interrupts when four NPU kernel symbols exist?

The binary runtime may split a named kernel into internal sublayers or use a schedule not visible in source. The defensible observation is five acknowledged hardware completions per inference; do not invent an unverified mapping.

39. Does the interrupt version run faster?

No. It measured about 4.9% slower end-to-end than polling in the direct comparison because of interrupt/kernel overhead. Its benefit is bounded blocking and improved schedulability, not raw latency.

Fault tolerance and concurrency

40. Why is the current application single-threaded?

One request at a time keeps protocol order, timing, recovery, and the binary runtime's global state deterministic. NPU hardware and ISR still operate concurrently with the blocked main thread.

41. Why is the NPU timeout 100 ms?
Normal stages complete in hundreds of microseconds, so 100 ms is comfortably above expected jitter while still bounding failure. It is a reliability threshold, not a performance target.

42. Why reset the full MCU after NPU timeout?
The binary runtime may retain partially advanced global continuation state. Without a documented local recovery contract, cold reset is the safest way to restore all software and peripheral state.

43. Why add a watchdog if you already have timeouts?
Software timeouts require software to keep executing. An independently clocked hardware watchdog also covers total application deadlock or broken control flow.

44. Why not feed the watchdog from a timer thread?
A background feeder could remain alive while useful processing is stuck. Feeding only at main-loop progress makes the watchdog reflect application liveness.

45. What is fault injection?
Deliberately causing a known failure—bad CRC, missing NPU start, watchdog starvation, UART BREAK—to prove detection and recovery paths that normal tests rarely execute.

Testing, WSL, and reproducibility

46. Why are unit tests not enough?
Host/native tests validate pure logic but cannot prove pin mux, PLL, NPU power, physical IRQ routing, watchdog reset, USB transport, or target timing. Hardware tests close those gaps.

47. What do ASan and UBSan add?
On native_sim they detect memory-safety errors and undefined C behavior in portable protocol code. They complement, not replace, target tests.

48. How do you prove the NPU code is linked?
CI inspects the final ELF symbol table for generated TINIElite kernels, the TI continuation handler, and project integration symbols. Runtime completion counters prove physical execution behavior.

49. How does hardware reach WSL?
`usbipd-win` attaches the Windows USB device to the WSL 2 VM. Linux then sees the XDS110/ACM interfaces; udev and the dialout group grant restricted access.

50. What does XDS110 do?
It is TI's on-board debug probe, exposing debug/programming and application UART interfaces over USB.

51. What does DSLite do?
TI UniFlash's command-line debug-server tool uses the CCXML target configuration to erase, program, verify, and control the device through XDS110.

52. Why use a clean west workspace?
The legacy Zephyr checkout contains older in-tree modifications and a duplicate board name. A clean pinned checkout proves the repository contains all required project changes and avoids accidental dependency on local state.

5. Hard challenges and honest answers

“You only tested six inputs. Why should I trust this?”

Trust is scoped. The six vectors establish deterministic integration regression, and repeating them 600 times stresses transport/runtime stability. They do not establish model quality. A true accuracy study needs a separate representative labeled dataset, provenance, and confusion-matrix metrics.

“Could the firmware simply return hardcoded expected values?”

The linked ELF contains generated NPU kernels; the runtime invokes project NPU memory callbacks; the physical device produces five acknowledged DONE interrupts per inference; a deliberately absent completion triggers the real timeout; and results contain timing/IRQ telemetry. Together these are stronger than output matching alone.

“Why should I believe 701 µs after earlier numbers were wrong?”

The earlier mistake was explicitly retained and corrected. Current measurements are paired with raw PLL/source/divider/flash register read-back proving 200 MHz. The lesson is that cycle conversion must be anchored to observed hardware state.

“Is this production ready?”

No; it is a hardware-validated engineering prototype. Production work would include async UART, deployment configuration without fault injection, wider peripheral/EMI/security testing, stack optimization, field update strategy, upstream review, and product-specific safety requirements.

“Why not compare against CPU inference?”

The available archive targets TINIElite and no equivalent source model or CPU-target artifact exists. A fair comparison requires the same model, quantization, preprocessing, compiler assumptions, and verified clock. Fabricating a substitute would be misleading.

“How much power did the NPU save?”

That was not measured. Latency and main-thread wait bounds were recorded, but energy requires board-current instrumentation and controlled workloads. Blocking the CPU does not by itself quantify system power.

“Why use raw registers rather than a standard Zephyr clock/watchdog driver?”

The preview platform lacked those reviewed upstream abstractions. The project isolated raw accesses, used devicetree addresses, followed TI SDK sequences, bounded status waits, and added read-back tests. Converting them into reusable upstream drivers is future work.

“What would you redesign with full model source?”

Reproduce the training/export toolchain, create an independent dataset evaluation, generate matched CPU/NPU variants, define a failure-reporting callback ABI, characterize per-layer latency/memory, and evaluate dynamic or partitioned model storage if the platform permits it.

6. STAR stories

6.1 Correcting the clock and performance evidence

Situation: The project reported impressive microsecond latency based on Zephyr cycle conversion.
Task: Verify that the measurement represented real hardware behavior.
Action: Added read-only clock telemetry, discovered MCLK was 32 MHz despite 200 MHz metadata, implemented TI's crystal/PLL/flash sequence, and added host validation of raw registers.
Result: Achieved read-back-validated 200 MHz operation and replaced misleading values with defensible 701 µs recovery-image timing.

6.2 Moving from polling to verified interrupts

Situation: The early NPU path polled a status register and was described too strongly.
Task: prove real interrupt-driven completion and bound failure.
Action: Implemented IRQ 41 status/clear/mask handling, atomic counters, semaphore delivery, 100 ms waits, protocol metrics, and spurious-interrupt rejection.
Result: 600 inferences produced exactly 3,000 acknowledged completions and zero spurious IRQs.

6.3 Hardening a fragile serial protocol

Situation: The prototype used a fixed packet and additive checksum and could desynchronize.
Task: create a recoverable test/control channel.
Action: Designed versioned length-delimited frames with sequences, CRC-32, typed errors, absolute deadlines, retries, and a sliding host parser.
Result: CRC corruption and UART BREAK were rejected, later transactions recovered, and stress testing required zero retries.

6.4 Recovering from complete deadlock

Situation: NPU timeouts could recover a peripheral stall but not a frozen application.
Task: add independent liveness supervision.
Action: Configured WWDT0 from a 32.768 kHz source, routed violation to BOOTRST, fed only on request-loop progress, and added deliberate starvation/status checks.
Result: Hardware reset occurred with cause `0x0e`, the watchdog restarted, and 600 post-recovery inferences passed.

6.5 Completing a native WSL workflow

Situation: The inherited project had stale Windows paths, a lost virtual environment, and an assumption that flashing had to remain in Windows.
Task: establish one reproducible Linux workflow under WSL.
Action: rebuilt the west/toolchain environment, attached XDS110 with usbipd, added restricted udev rules, installed Linux UniFlash, and wrote repository-relative scripts.
Result: Build, flash, verify, UART test, fault tests, and CI now operate from the WSL-centered repository.

7. Résumé wording and ownership

Strong concise bullet

> Built out-of-tree Zephyr support and a WSL-native deployment/test workflow for TI's AM13E230 Cortex-M33 MCU; integrated a TVM-generated TINIElite NPU waveform classifier with interrupt-driven completion, CRC-framed UART, bounded recovery, and hardware-backed CI evidence.

Evidence-oriented second bullet

> Validated 600 repeated on-board inferences with 3,000 acknowledged NPU DONE interrupts, zero spurious interrupts or transport retries, read-back-verified 200 MHz clocking, CRC/UART fault recovery, NPU timeout reset, and independent hardware-watchdog recovery.

Ownership sentence

“The neural network was supplied as a generated binary artifact. I owned the embedded platform bring-up, driver/runtime integration, host protocol, reliability mechanisms, WSL workflow, testing, evidence, and repository engineering.”

8. Live demonstration plan

8.1 Before the interview

- Charge laptop and connect the LaunchPad with a known data-capable USB cable.
- In elevated PowerShell, confirm the XDS110 BUSID and attach it to WSL.
- In WSL, verify `lsusb`, the serial device, and group permissions.
- Activate the clean west workspace environment and confirm UniFlash path.
- Build once and keep the known-good HEX.
- Keep JSON evidence available in case hardware connection fails during the interview.

8.2 Demonstration sequence

```
./scripts/build.sh
./scripts/flash.sh
# Press RESET once if READY is not observed after flashing.
./scripts/test.sh
./scripts/test.sh --clock-status
./scripts/test.sh --fault-test --repeat 10 --quiet
```

Explain what each command proves. Do not run watchdog/NPU reset tests unless time permits and you have practiced the reset/READY sequence.

8.3 If the board does not respond

1. Check `usbipd list`; reattach if state is only Shared.
2. Check the Linux serial path and permissions.
3. Ensure no terminal has the ACM port open.
4. Press RESET once and wait for `READY`.
5. Reflash and verify if necessary.
6. Fall back to committed evidence and explain the diagnostic process calmly.

9. Final fact sheet

Item	Value
Board / SoC / CPU	TI LP-AM13E230 / AM13E23019 / Arm Cortex-M33
Accelerator	TINIElite NPU
RTOS	Pinned development revision of Zephyr (commit recorded in `west.yml`)
Input / output	256 signed int8 values / 3 signed int8 scores
Classes	Sawtooth, sine, square
Clock	200 MHz MCLK; 100 MHz UART source; 25 MHz crystal
UART	115200 baud, 8N1, polling byte API
NPU interrupt	IRQ 41, five completions per inference
NPU stage timeout	100 ms
Hardware watchdog	WWDT0, 32.768 kHz source, four-second timeout
Protocol	Version 2, little-endian, sequence + length + IEEE CRC-32
Current recovery-image timing	554 µs wait, 129 µs runtime, 701 µs total median
Stress evidence	600/600 exact fixtures, zero retries, 3,000 DONE IRQs, zero spurious
Footprint	25,492 B flash; 43,616 B RAM including 32 KiB main stack
Model ownership	Generated artifact supplied externally; training/source model unavailable

Commands worth remembering

```
./scripts/build.sh                  # clean target build
./scripts/flash.sh                  # Linux DSLite erase/program/verify
./scripts/test.sh                   # all six fixtures
./scripts/pipeline.sh               # build + flash + test
./scripts/static_check.sh           # C formatting + Cppcheck
python -m unittest discover -s tests -v
ruff check tools tests
ruff format --check tools tests
shellcheck scripts/*.sh
```

> **FINAL MENTAL MODEL**
>
> The model archive supplies neural-network computation. This project supplies the trustworthy embedded system around it: a supported platform, known clocks, safe hardware access, validated transport, interrupt synchronization, bounded failure behavior, independent recovery, reproducible tooling, and evidence.